

Relatório de Implementação

Internacionalização (i18n) — 6 Idiomas ONU

Data: 26/05/2026 | Gerado por: Claude Code

1. Resumo Executivo

Foi implementado suporte completo a internacionalização (i18n) no aplicativo **Crisis Reporter** utilizando as bibliotecas **i18next** e **react-i18next**. A implementação cobre os seis idiomas oficiais da Organização das Nações Unidas (ONU): inglês, árabe, chinês, francês, russo e espanhol.

Todas as strings visíveis ao usuário foram extraídas dos componentes e substituídas por chamadas à função de tradução t('chave'). A lógica de negócio, o layout e o design system permaneceram **inalterados**.

Métrica	Valor
Idiomas implementados	6 (EN, AR, ZH, FR, RU, ES)
Arquivos de locale criados	6 arquivos JSON
Chaves de tradução por idioma	~120 chaves
Componentes / telas atualizados	11 arquivos
Novos erros TypeScript introduzidos	0
Dependências adicionadas	i18next, react-i18next

2. Idiomas Implementados

Os seis idiomas oficiais da ONU foram implementados com traduções contextualizadas para aplicações de resposta a crises humanitárias:

Código	Idioma	Script	Direção	Plurais especiais
en	English	Latim	LTR	one / other
ar	🟼🟼🟼🟼🟼 (Árabe)	Árabe	RTL ★	one / other
zh	🟼🟼 (Chinês)	Hanzi	LTR	other (sem plural)

fr	Français	Latim	LTR	one / other
ru	■■■■■■■■ (Russo)	Cirílico	LTR	one / few / many / other
es	Español	Latim	LTR	one / other

★ O árabe é o único idioma RTL (Right-to-Left). A diretividade é aplicada automaticamente via `document.documentElement.dir = 'rtl'` ao selecionar o árabe, e revertida para 'ltr' nos demais idiomas.

3. Arquivos Criados

Arquivo	Descrição
<code>src/i18n/index.ts</code>	Configuração i18next: carrega locais, detecta idioma do <code>localStorage</code> , aplica RTL
<code>src/i18n/locales/en.json</code>	Traduções em inglês (idioma padrão)
<code>src/i18n/locales/ar.json</code>	Traduções em árabe (com setas invertidas para RTL)
<code>src/i18n/locales/zh.json</code>	Traduções em chinês simplificado
<code>src/i18n/locales/fr.json</code>	Traduções em francês
<code>src/i18n/locales/ru.json</code>	Traduções em russo (com formas plurais completas)
<code>src/i18n/locales/es.json</code>	Traduções em espanhol neutro
<code>src/components/LanguageSelector.tsx</code>	Componente seletor de idioma com ícone de globo

4. Arquivos Modificados

Os arquivos abaixo tiveram apenas strings substituídas por chamadas `t('chave')`. Nenhuma lógica, layout ou estilo foi alterado.

Arquivo	Tela / Componente	Strings traduzidas
<code>screens/citizen/CameraScreen.tsx</code>	Câmera (Passo 1 de 5)	8
<code>screens/citizen/LocationScreen.tsx</code>	Localização (Passo 2 de 5)	10
<code>screens/citizen/ClassificationScreen.tsx</code>	Classificação (Passo 3 de 5)	30+
<code>screens/citizen/DetailsScreen.tsx</code>	Detalhes (Passo 4 de 5)	20+
<code>screens/citizen/ReviewScreen.tsx</code>	Revisão (Passo 5 de 5)	18
<code>screens/citizen/ConfirmationScreen.tsx</code>	Confirmação	8
<code>screens/agent/DashboardScreen.tsx</code>	Dashboard do Agente	1

components/agent/FilterPanel.tsx	Painel de Filtros	25+
components/agent/ExportButton.tsx	Botão de Exportação	5
components/agent/ObservationDetail.tsx	Detalhe da Observação	20+
components/map/ObservationPopup.tsx	Popup do Mapa	5
App.tsx	Raiz da aplicação	—

5. Estrutura de Chaves de Tradução

Todos os idiomas usam um único namespace (translation) com seções organizadas por contexto de uso:

Seção	Conteúdo	Exemplo de chave
common.*	Strings reutilizadas (Voltar, Próximo)	common.back
camera.*	Tela da câmera	camera.take_photo
location.*	Tela de localização	location.gps_ok
classification.*	Tela de classificação (danos, infra, crise)	classification.damage_partial
details.*	Tela de detalhes	details.need_food
review.*	Tela de revisão	review.submit
confirmation.*	Tela de confirmação	confirmation.received
dashboard.*	Dashboard do agente	dashboard.total_reports
export.*	Botão de exportação	export.geojson
observation.*	Popup e painel de detalhe	observation.conf
lang.*	Nomes dos idiomas	lang.ar
enum.*	Valores de enums (lookup dinâmico)	enum.damage_minimal

5.1 Padrão de Lookup de Enums

Valores de enums dinâmicos (nível de dano, tipo de infraestrutura, subtipo de crise, etc.) são traduzidos via concatenação de chave com o valor do enum:

```
t(`enum.damage_${data.damageLevel}`) // "minimal" → "Minimal" / "■■■■" / "■■"
t(`enum.infra_${data.infrastructureType}`) // "residential" → "Residencial" /
"■■■■" t(`enum.method_${data.locationMethod}`) // "manual_pin" → "Manual pin" /
"■■■■" t(`enum.need_${need}`) // "food" → "Food" / "■■■■" / "■■"
```

6. Componente LanguageSelector

O componente LanguageSelector foi criado em `src/components/LanguageSelector.tsx` e integrado ao `App.tsx`.

- Posição fixa (`position: fixed`) no canto superior direito — visível em todas as telas
- Ícone de globo SVG com nome do idioma atual exibido
- Dropdown com os 6 idiomas ao clicar
- Seleção persiste em `localStorage` (chave: `lang`)
- Ao trocar idioma: chama `i18n.changeLanguage()` + atualiza `document.documentElement.dir` e `.lang`
- RTL ativado automaticamente para árabe; LTR restaurado para demais idiomas
- Dropdown fecha ao clicar fora (handler em `mousedown`)

7. Pluralização

A biblioteca `i18next` gerencia pluralização via sufixo `_one` / `_other` (e variantes adicionais para idiomas com regras complexas):

Idioma	Sufixos suportados	Exemplo
English	<code>_one</code> , <code>_other</code>	1 report / 5 reports
Arabic	<code>_one</code> , <code>_other</code>	1 █████ / 5 █████
Chinese	<code>_other</code>	5 ███
French	<code>_one</code> , <code>_other</code>	1 rapport / 5 rapports
Russian	<code>_one</code> , <code>_few</code> , <code>_many</code> , <code>_other</code>	1 █████ / 3 █████ / 11 █████
Spanish	<code>_one</code> , <code>_other</code>	1 informe / 5 informes

8. Verificação de Qualidade

8.1 TypeScript

Executado `npx tsc --noEmit` após a implementação:

```
TypeScript: No errors found
```

Os 10 erros de build pré-existent (tsc -b) foram confirmados como anteriores à implementação — presentes no commit inicial 99232b3. Nenhum erro novo foi introduzido.

8.2 Erros pré-existent (não introduzidos)

Os seguintes erros existiam antes desta implementação e **não foram alterados**:

- `InfrastructureType` não inclui `school`, `health_center`, `bridge`, `power_station` — usados em `DEMO_OBSERVATIONS` e `FilterPanel`

- ClusterLayer.tsx: namespace leaflet.MarkerCluster não exportado pelos @types/leaflet

9. Decisões Técnicas

Namespace único

Todos os idiomas usam um único namespace translation, simplificando a configuração e evitando imports adicionais nos componentes.

Arrays de opções dentro do componente

Arrays com labels traduzíveis (INFRA_OPTIONS, ELECTRICITY_OPTIONS, etc.) foram movidos para dentro das funções de componente para serem recomputados quando o idioma mudar.

Seção enum.* para lookups dinâmicos

Valores de enums exibidos dinamicamente (ex: damage_level vindo da API) são traduzidos via padrão t(`enum.{categoria}\_{valor}`) em vez de mapeamentos hardcoded por componente.

Setas de navegação no árabe

As setas direcionais nas strings de navegação foram invertidas nas traduções árabes (ex: 'Próximo →' vira '← ■■■■■') para compatibilidade visual com RTL sem alterar CSS.

localStorage para persistência

O idioma selecionado persiste em localStorage com a chave lang. Na inicialização, src/i18n/index.ts lê esse valor e aplica o idioma e a diretividade correspondentes.

Import de efeito colateral em App.tsx

O i18n é inicializado via import './i18n' no App.tsx — import de efeito colateral — garantindo que a inicialização ocorra antes de qualquer renderização.

10. Como Usar

10.1 Adicionar tradução em novo componente

```
import { useTranslation } from 'react-i18next'; export default function
MeuComponente() { const { t } = useTranslation(); return {t('minha.chave')}; }
```

10.2 Adicionar novo idioma

- Criar src/i18n/locales/xx.json com todas as ~120 chaves
- Importar e registrar em src/i18n/index.ts: import xx from './locales/xx.json'
- Adicionar { code: 'xx', label: 'Nome' } ao array LANGUAGES em LanguageSelector.tsx

10.3 Testar localmente

```
npm run dev # Clicar no globo no canto superior direito para trocar idioma
```

